

Assignment 1 – Solutions

Database Management Systems · Weeks 1 & 2

Part I: Short Answer Questions

Q1. DBMS vs. Traditional File System

A traditional file system stores data as raw files managed by the operating system. Each application that uses the data must implement its own logic for reading, writing, searching, and controlling access. This leads to data redundancy, inconsistency and tight dependencies

A Database Management System (DBMS) provides a structured, centralised layer between applications and stored data. It handles concurrent access, enforces integrity constraints and supports querying (e.g. SQL)

Concrete scenario – Hospital patient records:

A hospital where multiple departments (emergency, pharmacy, billing) need simultaneous read/write access to patient records is a clear case for a DBMS. With a file system each department would maintain its own copy of patient data, leading to inconsistencies (e.g. conflicting medication lists). A DBMS stores a single authoritative copy, enforces access control per role (doctors vs. billing staff), and ensures that two nurses updating the same record concurrently do not corrupt each other's changes via transaction management.

Q2. Weak Entity vs. Strong Entity in an ER Diagram

A strong entity has its own primary key that uniquely identifies every instance of that entity without reference to any other entity.

A weak entity lacks a primary key of its own. It can only be uniquely identified in combination with the primary key of a related strong entity (its *owner/identifying entity*).

The primary key of a weak entity is the composite of: Primary key of owner entity + Partial key of the weak entity

For example, if **Dependent** is a weak entity of **Employee**, and the discriminator is **dep_name**, then the primary key of **Dependent** is (**emp_id**, **dep_name**).

The relationship connecting a weak entity to its owner is called an identifying relationship and is drawn with a double diamond in an ER diagram.

Q3. Phone Number Attribute Type for Students

Phone number should be modelled as a **multivalued attribute**.

Justification:

Because one student can hold zero, one, or many phone numbers, modelling it as multivalued is efficient.

Q4. Candidate Key vs. Super Key

Super key: Any set of one or more attributes whose values are guaranteed to uniquely identify each tuple(row) in a relation.

Candidate key: It is a minimal super key i.e. a super key with no redundant columns, which still identifies each row uniquely.

Relation: Student(roll_no, email, name, branch)

Candidate keys:

- roll_no – assigned uniquely to every student.
- email – each student has a unique institutional email address.

Super keys that are *not* candidate keys :

- {roll_no, email} – removing either of the attributes give a unique candidate key.
- {roll_no, name} – roll_no alone suffices.

Q5. Inserting a Row with a Non-existent student_id

- **Constraint violated:** Referential Integrity Constraint
- **What it protects against:** It ensures that a value in the student_id column of the Enrollment table always corresponds to an existing row in the Student table. Without this constraint, enrollment records could reference students who do not exist.
- **What should happen:** The DBMS should reject and roll back the insert operation, and also raise an ERROR.

Q6. Mapping an M:N Relationship to a Relational Schema

- **Additional structure required:** A new junction table must be created to represent the M:N relationship.
- **Primary key:** It consists of the **combination of the primary keys of both participating entities** (i.e. a composite primary key).
- **Why a single column in one table is insufficient:** In a relational table each cell holds a single atomic value. If we tried to store the M:N relationship as a column in, we would need multiple values in one cell, violating the First Normal Form. A junction table avoids this by creating one row per (student, course) pair, keeping every cell atomic.

Part II: Long Answer Question – Online Food Delivery Platform

Part A – Entity and Relationship Identification

Entities and key attributes:

Entity	Primary Key
Customer	customer_id
Restaurant	restaurant_id
MenuItem	item_id
Order	order_id

Relationships, cardinalities, and justifications:

Relationship	Cardinality	Justification
Customer – Order	1:N	A customer can place many orders, but each order belongs to exactly one customer.
Restaurant – Order	1:N	Each order is placed at exactly one restaurant, but a restaurant fulfils many orders over time.
Order – MenuItem	M:N	A single order can include many menu items, and a menu item can appear in many different orders.
Restaurant – MenuItem	1:N	A restaurant offers many menu items, and each menu item belongs to exactly one restaurant.

Part B – Attribute Classification

Multivalued attribute – phone of Customer:

A customer may register more than one contact number (e.g. personal and work). Because a single entity instance (**Customer**) can have a *set* of values for the same attribute, **phone** qualifies as a **multivalued attribute**.

Composite attribute – address of Customer (or Restaurant):

An address is naturally composed of distinct sub-parts: **street**, **city**, **state**, **pincode**. Because it can be decomposed into smaller components, **address** qualifies as a **composite attribute**.

Part C – Relational Schema for the Junction Table

The M:N relationship between **Order** and **MenuItem** requires a junction table. Each order can contain multiple items, each with its own quantity:

OrderItem(order_id, item_id, quantity)

- **Primary key:** (order_id, item_id) – the composite of both foreign keys uniquely identifies which item appears in which order.
- **Foreign keys:**
 - * order_id → Order(order_id)
 - * item_id → MenuItem(item_id)
- **Additional attribute:** quantity – records how many units of a given menu item are included in the order.

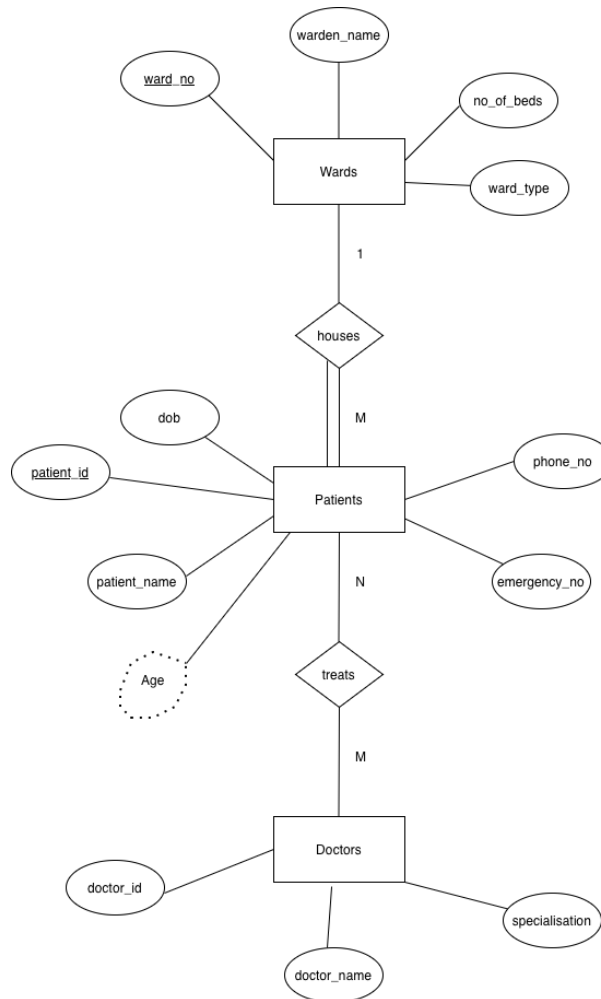


Figure 1: Hospital ER Diagram

Part III: ER Diagram Design

Question 1: Hospital ER Diagram

Question 2: Design Justification

1. Doctor–Patient M:N relationship and its associative entity:

The Doctor–Patient relationship is **M:N** because a single patient may be treated by multiple doctors during their hospital stay, while each doctor simultaneously treats many different patients.

This M:N relationship creates an **associative entity** called **Treats**. Its composite primary key consists of:

(doctor_id, patient_id)

2. Participation constraints:

- **Patient in *Houses* (Ward–Patient): Total participation.** Every patient in the hospital database is admitted to a ward. There is no reason to store a patient

record without a ward assignment. Hence **Patient** participates *totally* (drawn with a double line).

- **Ward in *Houses* (Ward–Patient): Partial participation.** A ward may exist in the system before any patients are assigned to it (e.g. some wards are not assigned to patients due to low occupancy). Therefore **Ward** participates only *partially* in this relationship.

3. Attribute design decision

The **age** attribute of **Patient** was modelled as a **derived attribute** rather than a stored attribute, with **dob** (date of birth) as the base attribute from which it is computed. Storing **age** as a static value would cause it to become stale over time — a patient’s age changes every year, so any stored value would require constant manual updates. By deriving **age** from **dob** at query time, the database always returns the correct current value without redundancy. In an ER diagram, a derived attribute is denoted by a **dashed ellipse**.